

베타테스트 자동화 도구

구현 계획서

Implementation Plan v2.0

초기 1인 개발 + 향후 팀 확장 대비 구조

대상: SIZL Agentic Brain 베타테스트 운영

포함 내용: 컴포넌트 구성 · 기술 스택 · 주차별 계획

작성일: 2026년 5월 19일 | 버전: 2.0

0. 이 문서를 읽는 방법

이 문서는 베타테스트 자동화 도구의 실제 개발을 시작하기 위한 작업 단위 분해본이다. 코드는 포함하지 않으며, 각 작업의 입력·출력·완료 기준에 집중한다.

주요 전제는 다음과 같다.

- 초기 개발자는 1 인이지만, 시스템은 향후 팀 합류와 확장이 가능하도록 설계한다.
- Backend 인프라를 두되, 처음에는 단일 서버에서 시작하여 점진적으로 분리한다.
- 확장 포인트(파란 박스로 표시)는 지금 만들지 않지만, 나중에 만들 자리를 비워둔다.
- 사내 전용 시스템으로, 외부 노출은 최소화한다.

문서 구성은 다음과 같다.

- 1 장: 결정 사항 — 시작 전 합의해야 하는 정책 결정
- 2 장: 컴포넌트 구성 — 시스템을 구성하는 6 개 컴포넌트
- 3 장: 기술 스택 — 컴포넌트별 채택 기술과 선정 사유
- 4 장: 디렉토리 구조 — 향후 확장 시 모듈 분리가 쉬운 모노레포 구조
- 5 장: 사전 준비 (W0) — 계정·키·환경 셋업
- 6~9 장: 주차별 작업 (W1~W4) — 각 작업의 입출력과 완료 기준
- 10 장: 통합 검증 시나리오
- 11 장: 배포 및 운영
- 12 장: 향후 확장 시나리오 — 팀 합류 시점에 무엇을 분리할지
- 부록: 체크리스트, 의사결정 로그 템플릿

참고. '확장 포인트'라고 표시된 부분은 MVP에서는 단순하게 구현하되, 나중에 분리·교체가 쉬운 구조로 만든다는 의미다. 처음부터 복잡하게 만들지 않는 것이 1 인 개발의 핵심이다.

1. 사전 결정 사항

구현 시작 전 확정해야 하는 정책 결정이다. 각 결정은 코드 구조에 영향을 미친다.

#	결정 항목	선택안 (확정 또는 권장)	재검토 시점
D1	Backend 언어/프레임워크	Python + FastAPI 확정	팀 합류 시 재논의
D2	초기 구조 복잡도	MVP 우선 — 단일 서버 시작, 모듈 분리 는 미리 설계	부하 증가 시
D3	GitHub 연동 방식	GitHub App + Webhook (1 차). Polling fallback 은 W4 이후 추가	Webhook 안정성 검증 후
D4	저장소 공개 범위	Private 권장 (테스트 계정·이메일 노출 방지)	운영팀 + 보안 합의
D5	LLM 자동 PR 생성 범위	모든 beta-feedback (P4 제외). confidence 0.5 미만 시 코멘트만	머지율 측정 후
D6	배포 환경	Cloud Run (Backend) + Vercel (Frontend) — 사내 IP 제한	팀 확장 시 K8s 검토
D7	DB 사용 여부	MVP 는 DB 없이 시작 (GitHub Issue 가 SoT, Sheet 가 미리). W4 에 SQLite 추가 (캐시·이벤트 로그)	PostgreSQL 전환 시점은 행 수 기준
D8	Message Queue	MVP 에 도입하지 않음. 비동기는 FastAPI BackgroundTasks 로 처리. Redis/Celery 는 W4 이후	LLM 호출 동시 처리량 증가 시

2. 컴포넌트 구성

2.1 컴포넌트 개요

시스템은 6 개 컴포넌트로 구성된다. 그중 4 개(Backend, Frontend, Scheduler, Issue Template)는 우리가 만들고, 2 개(LLM API, Google Sheet)는 외부 의존성이다.

#	컴포넌트	책임	기술 스택	MVP → 향후
C1	Web Frontend	폼 렌더링, 입력 검증, 미리보기, 이미지 업로드	Next.js + Tailwind	그대로 유지. 단, 인증을 SSO 로 확장
C2	Backend API	폼 수신, GitHub Issue 생성, Webhook 수신, 이슈/PR/Sheet 오케스트레이션	FastAPI + Pydantic + httpx	API 서버와 Worker 로 분리 가능
C3	LLM Service	이슈 분석 → diff 생성	Anthropic Claude API	Provider 추상화로 교체 가능
C4	Scheduler	매일 17 시 cron, Daily Snapshot 생성	GitHub Actions cron (MVP) → APScheduler (확장 시)	Backend 내장 또는 외부 분리
C5	Sheet Service	Google Sheet append/update	google-api-python-client	Backend 모듈로 시작, 별도 서비스 가능
C6	GitHub Repo	Issue/PR 데이터의 단일 진실 출처	GitHub App + Webhook	그대로

2.2 컴포넌트 간 데이터 흐름

MVP 단계의 흐름은 다음과 같다.

1. Frontend(C1) → Backend(C2): 폼 데이터를 JSON 으로 전송
2. Backend(C2) → GitHub(C6): Issue 생성 API 호출
3. GitHub(C6) → Backend(C2): Webhook 으로 issues.opened 이벤트 푸시

4. Backend(C2) → Sheet(C5): Raw Issues 시트에 행 추가
5. Backend(C2) → LLM(C3): 분석 요청
6. Backend(C2) → GitHub(C6): 브랜치 생성 + PR 생성
7. GitHub(C6) → Backend(C2): Webhook 으로 PR 상태 변경 이벤트 푸시
8. Backend(C2) → Sheet(C5): PR 컬럼 갱신
9. Scheduler(C4): 매일 17 시 Backend 의 /sync 엔드포인트 호출
10. Backend(C2) → Sheet(C5): Daily Snapshot 행 추가

2.3 Backend 내부 모듈 구조

Backend 는 하나의 FastAPI 애플리케이션이지만, 내부적으로는 책임별 모듈로 분리한다. 이 모듈들은 나중에 별도 서비스로 분리할 때 그대로 끄집어낼 수 있도록 설계한다.

모듈	역할	확장 시 분리 가능성
api/	HTTP 엔드포인트 정의 (FastAPI 라우터)	그대로 유지. 그 자체가 API Gateway 역할
services/issue/	폼 → Issue Body 변환, Issue 생성	Issue Intake Service 로 분리 가능
services/llm/	LLM 호출 + diff 생성 + PR 작성	LLM Worker 로 분리 (가장 분리 가능성 높음 — CPU/시간 부담)
services/sheet/	Google Sheet 동기화	Sheet Sync Worker 로 분리 가능
services/github/	GitHub API 클라이언트 래퍼	그대로 유지 (공통 라이브러리)
webhooks/	GitHub Webhook 수신 + 분기	Queue 도입 시 메시지 발행자 역할로 변환
schedulers/	일일 동기화 실행 로직	외부 cron → APScheduler 로 내재화 가능
core/	설정, 로깅, 인증 미들웨어	공통 라이브러리로 유지
models/	Pydantic 스키마 + DTO	공통 라이브러리로 분리

확장 포인트. `services/` 폴더의 각 서비스는 인터페이스(`Abstract Base Class`)를 두고 구현을 분리한다. 나중에 `services/llm/`이 별도 `Worker`가 되어도 인터페이스는 그대로이므로, 호출 코드는 한 줄도 안 바뀐다.

2.4 미래 확장 시나리오 (참고용)

팀 합류 또는 부하 증가 시 다음 순서로 점진 확장한다. MVP에서는 모두 단일 서버로 시작한다.

단계	트리거 조건	분리 대상
Phase 1 (MVP)	초기 (1~3 개월)	단일 FastAPI 서버에서 모두 처리. 비동기는 <code>BackgroundTasks</code>
Phase 2	LLM 호출 동시성 5+ 또는 일일 이슈 100 건+	<code>services/llm/</code> 을 별도 <code>Worker</code> 로 분리. Redis Queue (Celery 또는 RQ) 도입
Phase 3	팀 합류 (3 인 이상)	API 서버와 Sheet Worker 분리. DB 를 SQLite → PostgreSQL
Phase 4	타 부서 확산 또는 멀티 저장소 운영	멀티테넌시 도입, K8s 배포, 모니터링 스택 추가

3. 기술 스택

3.1 채택 기술 요약

영역	채택 기술	버전 (참고)	선정 사유
Backend Framework	FastAPI	0.115+	타입 안전성, 자동 문서, 비동기 지원, 학습 자료 풍부
Backend Runtime	Python	3.12+	LLM·GitHub·Sheets SDK 모두 풍부
ASGI Server	uvicorn	0.32+	FastAPI 표준
Pkg/Dep 관리	uv	0.5+	poetry 대비 10 배 빠름, lock 파일 표준
HTTP Client	httpx	0.27+	requests 의 비동기 버전. 동기/비동기 모두 지원
Validation	Pydantic v2	2.9+	FastAPI 기본 통합. JSON Schema 자동 생성
GitHub SDK	PyGithub + httpx	PyGithub 2.5+	PyGithub 은 REST, 추가 호출은 httpx 직접 사용
LLM SDK	anthropic	0.40+	Claude API 공식 SDK. tool use 지원
Sheets SDK	google-api-python-client + gspread	최신	gspread 는 간단한 작업, google-api-python-client 는 batch
Frontend Framework	Next.js (App Router)	15+	React 기반, SSR 가능, Vercel 호스팅 최적화
Styling	Tailwind CSS	4+	빠른 프로토타이핑, 디자인 시스템 일관성
Form Handling	React Hook Form + Zod	최신	Zod 스키마를 Backend Pydantic 과 형태 일치 가능
Backend Hosting	Google Cloud Run	-	컨테이너 기반, 자동 스케일, 사내 VPC 연동 가능

영역	채택 기술	버전 (참고)	선택 사유
Frontend Hosting	Vercel	-	Next.js 최적화, 자동 배포, 무료 티어 충분
Secret 관리	Google Secret Manager	-	Cloud Run 과 통합. 회전·감사 추적 가능
로깅	structlog + Cloud Logging	structlog 24+	구조화된 JSON 로그, Cloud Console 에서 검색
에러 추적	Sentry (선택)	최신	프로덕션 에러 자동 알림
스케줄러	GitHub Actions cron	-	MVP 는 외부 cron. 추후 APScheduler 내재화 검토
테스트	pytest + pytest-asyncio	최신	Python 표준 + FastAPI 비동기 테스트
코드 품질	ruff + mypy	ruff 0.7+	ruff 가 black + flake8 + isort 를 대체. 10 배 빠름
CI/CD	GitHub Actions	-	코드 저장소와 통합. 라이브러리/빌드 캐싱 강력

3.2 확장 시 추가될 기술 (현 단계 미도입)

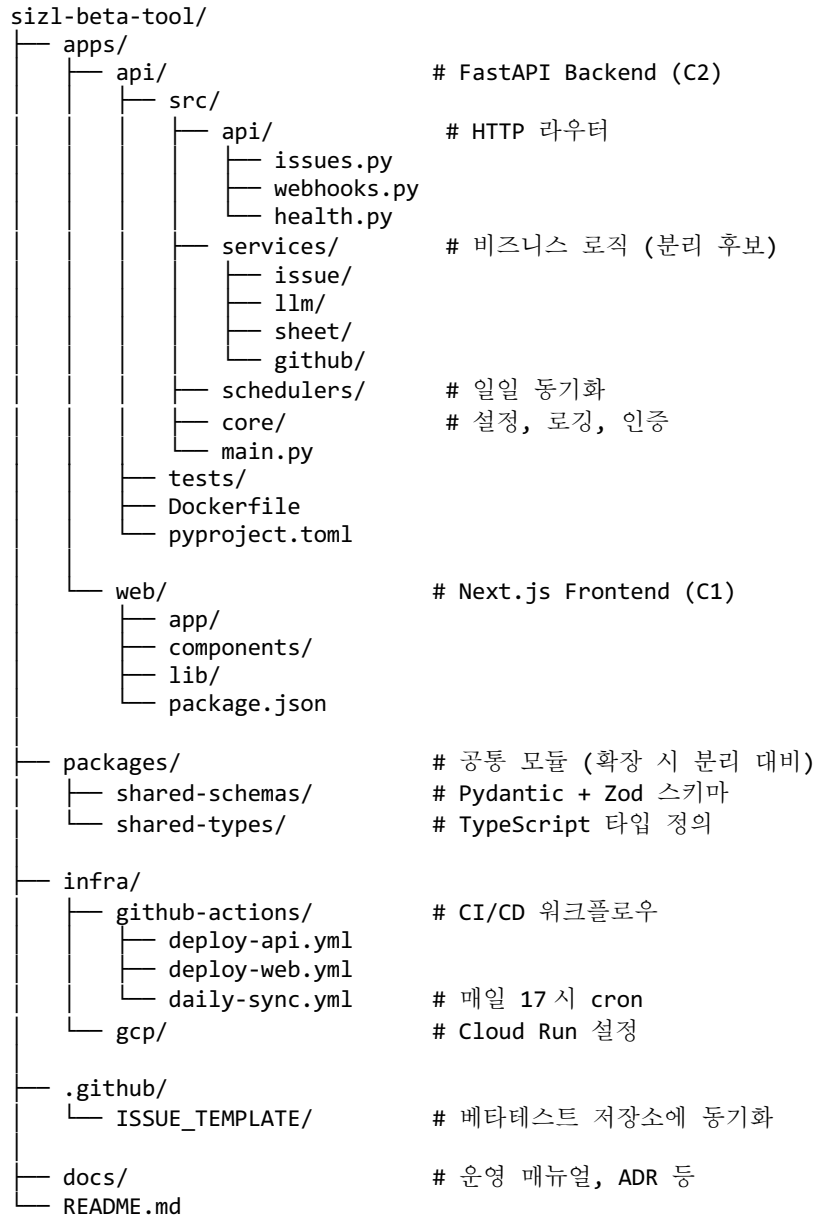
영역	후보 기술	도입 시점
Database	SQLite → PostgreSQL	W4 에 SQLite 도입 (이벤트 로그·캐시). 팀 합류 시 PostgreSQL
Message Queue	Redis + RQ 또는 Celery	LLM 동시 호출 5+ 시. 또는 LLM Worker 분리 시
ORM	SQLModel (FastAPI 친화)	DB 도입 시 함께
Migration	Alembic	DB 도입 시 함께
Metrics	Prometheus + Grafana	팀 합류 + 운영 가시성 요구 증가 시

영역	후보 기술	도입 시점
Distributed Tracing	OpenTelemetry	서비스 분리 후 (Phase 3+)

4. 디렉토리 구조

향후 모듈 분리가 쉽도록 모노레포 구조로 시작한다. apps/ 아래의 각 앱은 독립적으로 배포 가능하고, packages/ 아래의 공통 모듈을 공유한다.

4.1 전체 구조



확장 포인트. apps/ 아래에 각 앱이 독립된 Docker 이미지로 배포 가능하므로, 나중에 apps/llm-worker/, apps/sheet-sync/ 등을 추가하여 Backend 를 분리할 수 있다. packages/는 그때까지 공통 코드의 단일 출처가 된다.

4.2 모노레포 도구

- Python: uv workspaces 사용 (apps/api 에 별도 pyproject.toml)
- Node.js: pnpm workspaces 사용 (apps/web, packages/* 공유)
- CI: GitHub Actions 의 paths 필터로 변경된 앱만 빌드

5. W0: 사전 준비 (3 일)

코딩 시작 전 모든 계정·키·환경 셋업을 완료한다. 1인 개발이므로 여기서 막힘이 없어야 W1에 본격 집중할 수 있다.

5.1 GitHub 측 준비

#	작업	산출물	완료 기준
0.1	GitHub App 신규 등록	App ID, Private Key, Webhook Secret	저장소에 설치 완료
0.2	필요 권한 부여	Issues RW, PR RW, Contents RW, Metadata R	권한 명세 문서
0.3	Webhook 이벤트 구독	issues, pull_request 활성화. Webhook URL은 임시(ngrok)	ngrok 테스트로 이벤트 수신
0.4	라벨 체계 정비	priority:P1~P4, area:*, cause:*, dept:*, auto-pr-*	저장소 Labels 화면에서 확인
0.5	이슈 템플릿 업데이트	기존 2개 템플릿에 '테스트 환경' 추가	New Issue 화면에 노출 확인
0.6	브랜치 보호 규칙	main 보호, PR 필수, 리뷰 1인 이상	규칙 활성화 확인
0.7	자체 코드 저장소 생성	sizl-beta-tool 모노레포	초기 README + .gitignore

5.2 인프라 측 준비

#	작업	산출물	완료 기준
0.8	Google Cloud 프로젝트 생성	프로젝트 ID, 결제 활성화	Cloud Console 접근 가능
0.9	Cloud Run 활성화	Cloud Run + Cloud Build API	Hello World 컨테이너 1회 배포 성공
0.10	Secret Manager 활성화	Secret 1개 등록 테스트	Cloud Run에서 읽기 성공

#	작업	산출물	완료 기준
0.11	Sheets API 활성화 + 서비스 계정	서비스 계정 JSON 키	키 다운로드 + Secret Manager 등록
0.12	Google Sheet 생성	3 개 시트 (Raw / Snapshot / Dashboard)	서비스 계정에 편집자 권한 부여
0.13	Vercel 프로젝트 생성	프로젝트 + GitHub 연동	초기 deploy 1 회 성공
0.14	Anthropic API 키 발급	API 키, 월 한도 설정	첫 API 호출 성공
0.15	Slack 운영 채널 생성	Incoming Webhook URL	테스트 메시지 수신 확인
0.16	로컬 개발 환경 셋업	Python 3.12, Node 20, uv, pnpm, Docker	Hello World API + Web 로컬 실행

5.3 W0 완료 체크리스트

- ☐ 1 장의 D1~D8 결정 사항이 ADR 로 기록되었다 (docs/adr/)
- ☐ GitHub App 이 설치되어 ngrok 으로 Webhook 이 수신된다
- ☐ 라벨/템플릿이 정비되었다
- ☐ Google Sheet 가 생성되어 있고, 서비스 계정 권한이 부여되었다
- ☐ LLM API 키가 발급되어 첫 호출이 성공했다
- ☐ Cloud Run 에 Hello World 가 배포되었다
- ☐ Vercel 에 Hello World 가 배포되었다
- ☐ Slack 알림이 동작한다
- ☐ 로컬에서 모노레포 양쪽 앱(api, web)이 동시에 실행된다

6. W1: 기반 + 시트 + 웹 폼 골격

첫 주는 자동화 로직 없이 기반을 완성한다. (a) 모노레포 + CI/CD 파이프라인, (b) Google Sheet 템플릿, (c) 웹 폼 골격(제출 시 콘솔 로그).

6.1 작업 단위

#	작업명	내용	산출물	완료 기준
T1.1	모노레포 골격	apps/api, apps/web, packages/, infra/ 디렉토리 생성. 각 앱에 README	초기 커밋 + main 보호	두 앱이 로컬에서 실행됨
T1.2	FastAPI 스켈레톤	api/, services/, core/ 디렉토리. /health 엔드포인트만 작동. 설정은 pydantic-settings 로 환경변수 로드	Backend 기본 구조	/health 가 200 OK
T1.3	Service 인터페이스 정의	services/llm, services/sheet, services/github, services/issue 각각에 ABC(Abstract Base Class) 정의. 빈 구현체만 작성	인터페이스 + 빈 구현	import 시 에러 없음
T1.4	Pydantic 스키마	models/ 하위에 BugReport, EnhancementRequest, IssueMetadata 등 데이터 모델 정의. 두 템플릿 모두 커버	Pydantic 모델 + 단위 테스트	샘플 데이터로 validation 통과
T1.5	CI 파이프라인	GitHub Actions 에 lint + test + build 워크플로우. ruff/mypy/pytest. PR 마다 자동 실행	.github/workflows/ci.yml	Hello PR 이 CI 통과
T1.6	CD 파이프라인	main 머지 시 Cloud Run 자동 배포(Backend), Vercel 자동 배포(Frontend)	deploy-api.yml, deploy-web.yml	main 푸시 후 5 분 내 배포 완료

#	작업명	내용	산출물	완료 기준
T1.7	Raw Issues 시트 구성	기존 19 개 + 추가 10 개 컬럼 헤더. 드롭다운 데이터 검증 설정 (B/D/I/J/K/L/R/X)	Raw Issues 시트	운영자가 수동 1 행 입력 시 드롭다운 동작
T1.8	조건부 서식	K 열 우선순위 색상, R 열 처리 상태 색상, AA 열 7 일 이상 경고	서식 규칙 4 개	샘플 값 입력 시 색상 적용
T1.9	Daily Snapshot 시트	13 개 컬럼 헤더 + 더미 행 1 개	Snapshot 시트	헤더 + 더미 완성
T1.10	Dashboard 시트	KPI 6 개(병합셀+함수) + 차트 4 개(빈 데이터) + QUERY 액션 리스트	Dashboard 시트	더미 10 건으로 차트 의미 있게 표시
T1.11	시트 보호 설정	자동 컬럼 영역(A, C-K, Q-AC)을 서비스 계정만 편집 가능하게 보호	범위 보호 규칙	운영자 계정으로 보호 영역 편집 시 경고
T1.12	Next.js 폼 화면	이슈 리포트 / 개선 사항 두 화면. React Hook Form + Zod 검증. 이미지 미리보기. 제출 시 콘솔 로그만 (Backend 미연동)	Frontend 1 차 배포본	필수 누락 시 검증 오류, 완료 시 JSON 로그

6.2 W1 종료 시 모습

- Cloud Run 에 빈 Backend 가 배포되어 /health 가 응답
- Vercel 에 폼 화면이 배포되어 누구나 접근 가능 (아직 인증 없음)
- Google Sheet 는 운영자가 수동으로도 쓸 수 있는 완성 상태
- 폼 제출 시 데이터는 어디에도 저장되지 않음 (콘솔 로그만)

참고. 이 시점에 운영자에게 시트를 보여주고 더미 이슈 5 건을 입력시키면서 Dashboard 시트가 어떻게 시각화되는지 함께 확인한다. 운영자 피드백을 W2 작업 전에 반영하면 후속 작업이 줄어든다.

7. W2: 시트 동기화 (Backend ↔ GitHub ↔ Sheet)

두 번째 주는 가장 중요한 흐름을 완성한다. 웹 폼이 실제로 GitHub Issue 를 만들고, 그 Issue 가 시트로 자동 흘러가는 양방향 연결.

7.1 작업 단위

#	작업명	내용	산출물	완료 기준
T2.1	GitHub Service 구현	services/github/ 안에 App 인증, Issue/PR 생성, 라벨 부여 메서드. PyGithub + httpx 조합	GithubService 클래스	더미 저장소에 Issue 1 건 생성 성공
T2.2	폼→Issue Body 변환기	services/issue/builder.py. Pydantic 모델 → 정해진 마크다운으로 변환. YAML 헤더 + 기존 템플릿 구조	Builder + 단위 테스트	실제 이슈 3 건의 본문 재현
T2.3	POST /api/issues	폼 → Pydantic 검증 → Builder → GithubService 호출 → Issue 번호 반환	/api/issues 엔드포인트	Postman/curl 로 호출 시 Issue 생성
T2.4	Frontend 연동	폼 제출 시 /api/issues 호출, 결과 화면(Issue 번호+링크) 표시	Frontend 2 차 배포본	폼 제출 → Issue 등록 → 결과 표시
T2.5	이미지 업로드 처리	Frontend 가 이미지를 Base64 로 전송 → Backend 가 GitHub Issue 에 첨부 후 Markdown 링크로 본문 삽입	이미지 업로드 흐름	3 장 첨부 시 모두 본문에 표시
T2.6	Sheet Service 구현	services/sheet/. append_row, update_cells, find_row_by_issue_number 메서드	SheetService 클래스	더미 시트에 행 1 개 add 성공
T2.7	Webhook 엔드포인트	POST /api/webhooks/github.	/api/webhooks/github	GitHub 의 Webhook

#	작업명	내용	산출물	완료 기준
		HMAC 서명 검증 + 이벤트 분기 (issues, pull_request)		테스트 이벤트 200 OK
T2.8	issues.opened 핸들러	Webhook 에서 issues.opened 수신 → Sheet 에 새 행 추가 (자동 컬럼만)	핸들러 + 통합 테스트	Issue 생성 → 1 분 내 시트 행 등장
T2.9	재시도/에러 처리	httpx 에 지수백오프 추가. Sheets API 429 자동 재시도. 실패 이벤트는 구조화 로그 + Slack 알림	재시도 미들웨어	API 한도 강제 시 자동 복구
T2.10	운영자 데모	운영자 앞에서 폼→Issue→시트까지 시연. 운영자가 L~P 수작업 컬럼 입력	데모 승인서	운영자 OK → W3 진행

7.2 W2 의 위험 포인트

- 위험 1: 행 찾기 성능 — Issue 번호로 매번 전체 시트 스캔하면 느림. 메모리 캐시(번호→행 번호)를 둬. 캐시는 W4 일일 동기화에서 재구성
- 위험 2: 동시성 — 같은 이슈의 두 Webhook 이 동시 도착 가능. SheetService 의 update 는 멍등(idempotent) 설계
- 위험 3: 시트 보호 충돌 — 서비스 계정이 보호 영역을 못 쓰면 실패. T1.11 에서 예외 등록 재확인
- 위험 4: 이벤트 누락 — Cloud Run 콜드 스타트로 Webhook 일부 누락 가능. W4 일일 동기화에서 보강

확장 포인트. FastAPI BackgroundTasks 로 Sheet 동기화를 비동기 처리한다. 동시성이 5+ 가 되면 Celery + Redis 로 자연스럽게 이전 가능.

8. W3: LLM Worker (자동 PR 생성)

세 번째 주는 가장 까다롭다. LLM 품질에 따라 머지율이 결정되므로 프롬프트 튜닝에 시간을 충분히 배정한다.

8.1 작업 단위

#	작업명	내용	산출물	완료 기준
T3.1	이슈 파서	services/issue/parser.py. Issue Body 의 YAML 헤더 + 마크다운 섹션을 구조화 객체로 추출	Parser + 단위 테스트	실제 이슈 10 건 파싱 누락 없음
T3.2	코드 컨텍스트 수집기	services/llm/context.py. 테스트 영역 + 세부 기능으로 fuzzy 검색 → 관련 파일 5~10 개 선별	ContextCollector	토큰 한도 내 최적 파일 선별
T3.3	LLM Provider 추상화	services/llm/provider.py. LLMProvider ABC + AnthropicProvider 구현. 구조화 JSON 응답 강제	Provider 인터페이스 + 구현	샘플 호출로 JSON 응답 검증
T3.4	프롬프트 v1	services/llm/prompts/. 원인 분석 + diff 생성 요청. few- shot 예제 3 건	프롬프트 yaml	샘플 이슈 분석 결과 합리적
T3.5	응답 검증기	services/llm/validator.py. JSON 스키마 검증 + diff dry-run (git apply --check)	Validator	정상/비정상 diff 분류 정확
T3.6	PR Builder	services/llm/pr_builder.py. fix/issue-N 브랜치 생성 → 커밋 → PR 생성 (정형 본문)	PRBuilder	샘플 이슈로 PR 정상 생성
T3.7	issues.opened 에 LLM 연동	Webhook 핸들러에서 Sheet 추가 직후 BackgroundTask 로 LLM 호출 → PR 생성	통합 흐름	Issue 생성 → 5 분 내 PR 자동 등장

#	작업명	내용	산출물	완료 기준
T3.8	pull_request 핸들러	PR 생성/머지/닫힘 이벤트 → Sheet 의 W/X/Y/Z 갱신. 머지 시 R=완료, Q=오늘	PR 핸들러	PR 머지 후 시트 자동 반영
T3.9	프롬프트 튜닝	과거 이슈 10 건 회귀 테스트. 머지 가능 비율 측정하며 v2, v3 반복	프롬프트 v3 + 평가 리포트	10 건 중 3 건 머지 가능 수준
T3.10	실패 폴백	confidence < 0.5 / diff 적용 불가 / 토큰 초과 시 PR 생성 안 함. 코멘트만 + auto-pr- failed 라벨	폴백 로직	강제 실패 시 안전한 폴백

8.2 W3 의 위험 포인트

- 위험 1: LLM 출력 품질 — 머지율이 너무 낮으면 노이즈. 30% 미만이면 D5 재검토 (P1 만 처리하는 식으로 축소)
- 위험 2: 비용 폭주 — 이슈당 LLM 호출 비용 측정. 일일 한도 설정 + 도달 시 자동 중단 + Slack 알림
- 위험 3: 잘못된 PR 머지 — 브랜치 보호 + 사람 리뷰 필수가 안전망. PR 본문에 '⚠️ 자동 생성' 강조
- 위험 4: LLM 데이터 누출 — 이슈 본문의 이메일·계정 정보가 프롬프트에 포함됨. Anthropic 데이터 보존 정책 확인 + 마스킹 적용

확장 포인트. LLM 호출이 BackgroundTask 로 처리되지만, 동시성이 5+ 가 되면 services/llm/을 별도 Worker 로 분리하고 Redis Queue 로 연결한다. 인터페이스가 ABC 로 정의되어 있어 호출 코드는 변하지 않는다.

9. W4: 일일 보고 + 안정화 + 베타 오픈

마지막 주는 일일 보고 자동화와 전체 안정화, 그리고 베타 테스터 공개.

9.1 작업 단위

#	작업명	내용	산출물	완료 기준
T4.1	일일 통계 계산	schedulers/daily.py. Raw Issues 로부터 13 개 통계 계산	통계 함수	시트 함수와 결과 일치
T4.2	Snapshot 행 추가	계산 결과를 Daily Snapshot 시트에 append	append 로직	1 행 추가 + Dashboard 차트 반영
T4.3	전수 동기화	GitHub API 로 모든 Issue/PR 페이지네이션 조회 → 시트와 비교 → 누락/불일치 갱신	동기화 함수	시트 행 수동 삭제 → 복구됨
T4.4	POST /admin/sync 엔드포인트	동기화를 트리거하는 내부 엔드포인트. 인증 필수 (Service-to-Service)	/admin/sync	수동 호출 시 동기화 실행
T4.5	GitHub Actions cron	매일 17:00 KST 에 /admin/sync 호출하는 워크플로우. 타임존 명시	daily-sync.yml	3 일 연속 정시 실행
T4.6	Slack 알림	동기화 성공/실패 결과 + 에러 상세를 Slack 운영 채널로 전송	Slack 알림기	강제 실패 시 알림 도착
T4.7	이벤트 로그 DB	SQLite 도입. 모든 Webhook 이벤트 + LLM 호출 + Sheet 업데이트 기록 (감사·디버깅용)	SQLite + event_logs 테이블	실행 후 이벤트 조회 가능
T4.8	E2E 통합 테스트	폼→Issue→PR→머지→시트→일일 보고 흐름을 7 일치 더미 데이터로 검증 (10 장 시나리오)	E2E 리포트	시나리오 5 개 모두 통과
T4.9	운영 매뉴얼	docs/operations.md. 일상 점검 + 장애 대응 + Secret 회전 절차	운영 매뉴얼 md	운영자가 매뉴얼만 보고 1 주일 운영 가능

#	작업명	내용	산출물	완료 기준
T4.10	베타 가이드	테스터용 1 페이지 가이드 + FAQ	가이드 페이지	테스터 2 명이 가이드만 보고 첫 이슈 제출
T4.11	Soft Launch	내부 테스터 5 명에게만 먼저 공개. 3 일 운영	Soft Launch	3 일 무사고

10. 통합 검증 시나리오

W4 의 T4.8 에서 실행할 E2E 시나리오 5 개다.

#	시나리오	실행 절차	성공 기준
S1	정상 흐름	폼 제출 → Issue → 5 분 대기 → 자동 PR → 머지 → 17 시 동기화	시트 모든 컬럼 정상 채워짐. Snapshot 1 행 추가
S2	LLM 실패	의도적으로 모호한 이슈 제출	auto-pr-failed 라벨, 시트 Z=failed, 코멘트 첨부
S3	수작업 보호	시트 L~P 에 메모 입력 후 GitHub 라벨 변경	L~P 그대로, 자동 컬럼만 갱신
S4	동기화 복구	시트 행 1 개 수동 삭제 후 17 시 대기	자동 복구됨 + Slack 알림
S5	Webhook 누락	Cloud Run 일시 중단 → 이슈 생성 → 재가동 → 17 시 대기	일일 동기화로 누락분 보강

11. 배포 및 운영

11.1 Soft Launch 일정

- 11. D-Day: 내부 테스터 5 명 공개. 3 일 운영
- 12. D+3: 회고. 치명적 장애 없으면 다음 단계
- 13. D+4: 전체 베타 테스터 안내 메일
- 14. D+5~11: 첫 주 집중 모니터링
- 15. D+12: 운영 모드 전환

11.2 일상 운영 루틴

시각	담당	작업
09:00	개발자	Dashboard 확인 → P1 Open 즉시 처리
10:00	개발자	어제 생성된 자동 PR 리뷰
17:00	Cron 자동	Snapshot append + 전수 동기화 + Slack 알림
17:15	운영자	Slack 확인 + 통계 검토 + 이상치 조사
주 1 회	개발+운영	주간 회고 30 분

11.3 4 단계 롤백 절차

- Level 1 (Frontend): Vercel 에서 이전 배포로 즉시 rollback. 1 분 내 복구
- Level 2 (LLM Worker): D5 결정으로 LLM 호출 비활성화. 이슈 등록·시트 동기화는 정상
- Level 3 (Sheet Sync): Sheet Service 비활성화. 운영자가 수동 입력으로 복구
- Level 4 (전체): Backend 전체 중단. 베타 테스터는 GitHub Issue 직접 작성으로 안내

11.4 정기 점검

- 주 1 회: LLM API 사용량 (80% 초과 시 알림)
- 월 1 회: API 키 회전 (LLM, GitHub App, 서비스 계정)
- 월 1 회: 시트 행 수 점검 (1 만 행 넘으면 아카이브 검토)
- 분기 1 회: 베타 테스터 만족도 설문

12. 향후 확장 시나리오

MVP 출시 후, 팀 합류 또는 부하 증가 시점에 시스템을 단계적으로 확장한다. 각 단계는 1~2 주 작업으로 완료 가능하도록 설계되어 있다.

12.1 Phase 2: LLM Worker 분리 (트리거: 일일 이슈 100 건+ 또는 LLM 동시성 5+)

- 16. apps/llm-worker/ 추가. 기존 services/llm/ 코드를 그대로 이전
- 17. Redis 도입. apps/api 는 Redis Queue 에 작업 발행
- 18. apps/llm-worker 가 Queue 구독 + 처리
- 19. 인터페이스는 변하지 않으므로 호출 코드 변경 없음

12.2 Phase 3: 팀 합류 (트리거: 개발자 3 인 이상)

- 20. SQLite → PostgreSQL 이전. Alembic 으로 마이그레이션
- 21. apps/sheet-sync/ 분리
- 22. 코드 리뷰 프로세스 도입 (PR 템플릿, CODEOWNERS)
- 23. API 문서 강화 (FastAPI 자동 OpenAPI 를 docs.* 서브도메인에 호스팅)

12.3 Phase 4: 멀티 저장소 / 멀티 부서 (트리거: 다른 부서 도입)

- 24. 멀티테넌시 — 저장소별 설정 분리
- 25. K8s 이전 (GKE Autopilot 추천)
- 26. Prometheus + Grafana 모니터링
- 27. LLM Provider 추상화 확대 (OpenAI, 자체 모델 등 추가 가능)

12.4 확장 시점 식별 기준

지표	임계값	권장 액션
일일 이슈 수	100 건+	Phase 2 — LLM Worker 분리
LLM 동시 호출	5+ (동시)	Phase 2 — Queue 도입
Backend p95 응답시간	3 초+	프로파일링 → 병목 분석 → 분리 결정
운영 개발자 수	3 인+	Phase 3 — DB/문서 강화

지표	임계값	권장 액션
월 LLM 비용	계획 초과 시	D5 재검토 (처리 범위 축소)

13. 부록

13.1 전체 산출물 체크리스트

W0

- ☐ ADR 8 개 (D1~D8) 기록
- ☐ GitHub App 설치 + 라벨/템플릿 정비
- ☐ Google Sheet 빈 템플릿
- ☐ Cloud Run + Vercel Hello World 배포
- ☐ 모든 Secret 등록

W1

- ☐ 모노레포 골격 + CI/CD
- ☐ FastAPI 스켈레톤 + Service 인터페이스 정의
- ☐ Pydantic 스키마
- ☐ 시트 3 종(Raw/Snapshot/Dashboard) 완성
- ☐ Next.js 폼 골격 배포

W2

- ☐ GithubService + SheetService
- ☐ 폼→Issue Body Builder
- ☐ /api/issues + /api/webhooks/github
- ☐ issues.opened 핸들러
- ☐ 운영자 데모 통과

W3

- ☐ 이슈 파서 + 컨텍스트 수집기
- ☐ LLM Provider 추상화 + 프롬프트 v3
- ☐ 응답 검증기 + PR Builder
- ☐ pull_request 핸들러
- ☐ 머지율 30%+ 달성

W4

- ☐ 일일 동기화 + Snapshot
- ☐ /admin/sync + GitHub Actions cron

- ☐ Slack 알림 + SQLite 이벤트 로그
- ☐ E2E 5 개 시나리오 통과
- ☐ 운영 매뉴얼 + 베타 가이드
- ☐ Soft Launch 3 일 무사고

13.2 의사결정 로그 (ADR) 템플릿

- 결정 번호: D1
- 결정 일자: YYYY-MM-DD
- 결정 항목: (예: Backend 언어/프레임워크)
- 선택안: (예: Python + FastAPI)
- 고려한 대안: (예: Node.js + NestJS, Go + Fiber)
- 결정 사유: (LLM·GitHub·Sheets SDK 모두 풍부, 학습 자료 ...)
- 재검토 시점: (예: 팀 합류 시)
- 관련 문서: (다른 ADR 참조)

13.3 변경 이력

- v1.0 (이전 작성): GitHub Actions 단일 워크플로우 기반 단순화
- v2.0 (현재): Backend(FastAPI) 도입, 향후 팀 확장 대비 모듈 구조, MVP 우선 원칙 적용